BÀI TẬP CHƯƠNG 6 (Deadlocks)

Sinh viên có thể làm thêm bài tập ở sách tham khảo chính "Operating System Concepts", version 9, chương 7 (Deadlocks)

Để giải quyết deadlock, hệ điều hành dùng 3 cách:

- **Ngăn (Deadlock Prevention):**

  Không cho phép (ít nhất) 4 điều kiện cần deadlock xảy ra

- **Tránh (Deadlock Avoidance):**

  Hệ thống luôn phải ở trạng thái an toàn
  (Dùng giải thuật Banker để kiểm tra độ an toàn của hệ thống: tìm chuỗi an toàn, nếu có một chuỗi an toàn tồn tại → Hệ thống an toàn)

  - *Khi có một yêu cầu cấp phát mới, hệ thống sẽ kiểm tra nếu yêu cầu này được cấp phát thì hệ thống còn an toàn không (tìm chuỗi an toàn, nếu có một chuỗi an toàn tồn tại → Hệ thống an toàn)*
  - *Nếu hệ thống vẫn an toàn, thì yêu cầu này được cấp phát; nếu hệ thống không an toàn, thì yêu cầu này không được cấp phát*

- **Kiểm tra và Phục hồi (Deadlock detection and Recovery):**
  Cho phép hệ thống vào trạng thái deadlock, nhưng sau đó khi phát hiện deadlock (Detection) thì phục hồi lại hệ thống (Recovery)

Bài tập dạng hệ điều hành giải quyết deadlock theo kiểu tránh deadlock (Deadlock Avoidance) với giải thuật Banker:

1. Tại thời điểm xem xét, hệ điều hành có 5 process và 4 tài nguyên (A, B, C, D) với trạng thái như hình bên dưới:

	Allocation	*Max*	*Available*
	A B C D	*A B C D*	*A B C D*
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

Hệ điều hành giải quyết deadlock bằng cách tránh deadlock (Deadlock Avoidance) và dùng giải thuật Banker để kiểm tra độ an toàn của hệ thống. Trả lời các câu hỏi sau:

a. Tìm ma trận Need tương ứng
b. Hệ thống có an toàn không?
c. Nếu P_1 yêu cầu tài nguyên (0, 4, 2, 0), hỏi yêu cầu này có được đáp ứng không?

Đáp án:

a. Ma trận Need

	Need
P0	0, 0, 0, 0
P1	0, 7, 5, 0
P2	1, 0, 0, 2
P3	0, 0, 2, 0
P4	0, 6, 4, 2

b. Hệ thống an toàn
do tồn tại chuỗi an toàn

Đặt Work = Available = (1, 5, 2, 0)
Với Work = (1, 5, 2, 0) → Có thể cấp tài nguyên cần cho P0 hoặc P3. Giả sử cấp cho P0
Sau khi P0 thực thi xong, Allocation (0, 0, 1, 2) của P0 được trả cho hệ điều hành → Work = (1, 5, 3, 2)
Với Work = (1, 5, 3, 2) → Có thể cấp tài nguyên cần cho P2 hoặc P3. Giả sử cấp cho P2
Sau khi P2 thực thi xong, Allocation (1, 3, 5, 4) của P2 được trả cho hệ điều hành → Work = (2, 8, 8, 6)
Với Work = (2, 8, 8, 6) → Có thể cấp tài nguyên cần cho P1 hoặc P3, hoặc P4. Giả sử cấp cho P1
Sau khi P1 thực thi xong, Allocation (1, 0, 0, 0) của P1 được trả cho hệ điều hành → Work = (3, 8, 8, 6)
Với Work = (3, 8, 8, 6) → Có thể cấp tài nguyên cần cho P3 hoặc P4. Giả sử cấp cho P3.
Sau khi P3 thực thi xong, Allocation (0, 6, 3, 2) của P3 được trả cho hệ điều hành → Work = (3, 14, 11, 8)
Với Work = (3, 14, 11, 8), cuối cùng, P4 được cấp phát tài nguyên cần. Sau khi P4 thực thi xong, Allocation (0, 0, 1, 4) của P4 được trả cho hệ điều hành → Work = (3, 14, 12, 12)
⇨ **Chuỗi an toàn là: P0, P2, P1, P3, P4**

Chú ý: Còn các chuỗi an toàn khác, tổng cộng 36 chuỗi an toàn trong trường hợp này.

c.
Need P1 = (0, 7, 5, 0)
P_1 yêu cầu tài nguyên (0, 4, 2, 0)
(0, 4, 2, 0) < (0, 7, 5, 0): hợp lệ
Nếu yêu cầu của (0, 4, 2, 0) của P1 được cấp, trạng thái hệ thống sẽ chuyển thành:

	Allocation	*Max*	*Available*
	A B C D	*A B C D*	*A B C D*
P_0	0 0 1 2	0 0 1 2	**1 1 0 0**
P_1	**1 4 2 0**	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

	Need
P0	0, 0, 0, 0
P1	**0, 3, 3, 0**
P2	1, 0, 0, 2
P3	0, 0, 2, 0
P4	0, 6, 4, 2

Dùng giải thuật Banker kiểm tra xem hệ thống có an toàn không:
Work = Available = (1 1 0 0)

Với Work = (1, 1, 0, 0) → Có thể cấp tài nguyên cần cho P0
Sau khi P0 thực thi xong, Allocation (0, 0, 1, 2) của P0 được trả cho hệ điều hành → Work = (1, 1, 1, 2)
Với Work = (1, 1, 1, 2) → Có thể cấp tài nguyên cần cho P2
Sau khi P2 thực thi xong, Allocation (1, 3, 5, 4) của P2 được trả cho hệ điều hành → Work = (2, 4, 6, 6)
Với Work = (2, 4, 6, 6) → Có thể cấp tài nguyên cần cho P1 hoặc P3. Giả sử cấp cho P3
Sau khi P3 thực thi xong, Allocation (0, 6, 3, 2) của P3 được trả cho hệ điều hành → Work = (2, 10, 9, 8)
Với Work = (2, 10, 9, 8) → Có thể cấp tài nguyên cần cho P1 hoặc P4. Giả sử cấp cho P1.
Sau khi P1 thực thi xong, Allocation (1, 4, 2, 0) của P1 được trả cho hệ điều hành → Work = (3, 14, 11, 8)
Với Work = (3, 14, 11, 8), cuối cùng, P4 được cấp phát tài nguyên cần. Sau khi P4 thực thi xong, Allocation (0, 0, 1, 4) của P4 được trả cho hệ điều hành → Work = (3, 14, 12, 12)
⇨ **Chuỗi an toàn là: P0, P2, P1, P3, P4**
Vậy hệ thống vẫn an toàn nếu cấp phát cho P1 (0, 4, 2, 0) ➔ Yêu cầu này được đáp ứng.

Chú ý: Còn các chuỗi an toàn khác, tổng cộng 4 chuỗi an toàn trong trường hợp này.

2.

Cho hệ thống như sau (hệ điều hành giải quyết deadlock bằng cách Deadlock Avoidance)

	Allocation	*Max*
	A B C D	*A B C D*
P_0	3 0 1 4	5 1 1 7
P_1	2 2 1 0	3 2 1 1
P_2	3 1 2 1	3 3 2 1
P_3	0 5 1 0	4 6 1 2
P_4	4 2 1 2	6 3 2 5

a. Cho Available = (0, 3, 0, 1), sử dụng giải thuật Banker để xác định xem hệ thống có an toàn không?
b. Cho Available = (1, 0, 0, 2), sử dụng giải thuật Banker để xác định xem hệ thống có an toàn không?

Đáp án:

a. Không an toàn. Do không tồn tại một chuỗi an toàn nào.
(Need của P2, P1 và P3 có thể được đáp ứng, nhưng sau đó Need của P0 và P4 thì không được)
Sinh viên tự giải thích chi tiết như bài tập 1
b. An toàn.
Chuỗi an toàn có thể: P1, P2, P3, P0, P4 (Vẫn còn các chuỗi an toàn khác)
Sinh viên tự giải thích chi tiết như bài tập 1

3.

Cho hệ thống như sau (hệ điều hành giải quyết deadlock bằng cách Deadlock Avoidance)

	Allocation	*Max*	*Available*
	A B C D	*A B C D*	*A B C D*
P_0	2 0 0 1	4 2 1 2	3 3 2 1
P_1	3 1 2 1	5 2 5 2	
P_2	2 1 0 3	2 3 1 6	
P_3	1 3 1 2	1 4 2 4	
P_4	1 4 3 2	3 6 6 5	

Sử dụng giải thuật Banker, trả lời các câu hỏi sau:

a. Nếu P1 yêu cầu thêm (1, 1, 0, 0), yêu cầu này có được đáp ứng không?
b. Nếu P4 yêu cầu thêm (0, 0, 2, 0), yêu cầu này có được đáp ứng không?

Đáp án:

a. Được.
   Sinh viên tự giải thích tại sao như trong bài tập 1
b. *Sinh viên tự giải*